

✓ Lab 05 - K-Nearest Neighbors (KNN)

✓ Cell 1: Import libraries

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

✓ Cell 2: Load Iris dataset

```
# Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

print("Dataset Shape:", X.shape)
print("Target Classes:", iris.target_names)
```

```
Dataset Shape: (150, 4)
Target Classes: ['setosa' 'versicolor' 'virginica']
```

✓ Cell 3: Train-Test Split

```
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 120
Testing samples: 30
```

✓ Task 1: KNN Without Library Function

✓ Cell 4: Euclidean Distance

```
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((point1 - point2) ** 2))
```

✓ Cell 5: Implement KNN

```
def knn_predict(X_train, y_train, X_test, k=3):

    predictions = []

    for test_point in X_test:

        distances = []

        # Calculate distance from test point to every training point
        for i in range(len(X_train)):
            distance = euclidean_distance(test_point, X_train[i])
            distances.append((distance, y_train[i]))

        # Sort according to distance
        distances.sort(key=lambda x: x[0])

        # Select k nearest neighbors
        k_nearest = distances[:k]
```

```
# Get their class labels
labels = [label for distance, label in k_nearest]

# Majority voting
prediction = max(set(labels), key=labels.count)

predictions.append(prediction)

return np.array(predictions)
```

Cell 6: Test our KNN

```
# Implement KNN with k = 3
y_pred = knn_predict(X_train, y_train, X_test, k=3)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy * 100:.2f}%")
```

Model Accuracy: 100.00%

Task 2: Three Distance Measures

Cell 7: Distance Functions

```
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((point1 - point2) ** 2))

def manhattan_distance(point1, point2):
    return np.sum(np.abs(point1 - point2))

def minkowski_distance(point1, point2, p):
    return np.sum(np.abs(point1 - point2) ** p) ** (1 / p)
```

Formulas

Euclidean:

$$d = \sqrt{\sum |x_i - y_i|^2}$$

Manhattan:

$$d = \sum |x_i - y_i|$$

Minkowski:

$$d = \left(\sum |x_i - y_i|^p \right)^{1/p}$$

Minkowski is a general form where the value of **p** determines the distance.

Cell 8: KNN with Distance Selection

```
def knn_predict(X_train, y_train, X_test, k=3, distance_type="euclidean", p=3):
    predictions = []

    for test_point in X_test:
        distances = []

        for i in range(len(X_train)):
            if distance_type == "euclidean":
                distance = euclidean_distance(test_point, X_train[i])
            elif distance_type == "manhattan":
                distance = manhattan_distance(test_point, X_train[i])
```

```

        elif distance_type == "minkowski":
            distance = minkowski_distance(test_point, X_train[i], p)

        else:
            raise ValueError("Invalid distance type")

        distances.append((distance, y_train[i]))

    # Sort by distance
    distances.sort(key=lambda x: x[0])

    # Select k nearest neighbors
    k_nearest = distances[:k]

    # Extract labels
    labels = [label for distance, label in k_nearest]

    # Majority voting
    prediction = max(set(labels), key=labels.count)

    predictions.append(prediction)

return np.array(predictions)

```

✓ Cell 9: Take Distance as Input

```

print("Select Distance Measure:")
print("1. Euclidean")
print("2. Manhattan")
print("3. Minkowski")

choice = int(input("Enter your choice: "))

if choice == 1:
    distance_type = "euclidean"

elif choice == 2:
    distance_type = "manhattan"

elif choice == 3:
    distance_type = "minkowski"

else:
    print("Invalid choice")
    distance_type = "euclidean"

k = int(input("Enter value of K: "))

if distance_type == "minkowski":
    p = int(input("Enter value of p for Minkowski distance: "))
else:
    p = 3

y_pred = knn_predict(
    X_train,
    y_train,
    X_test,
    k=k,
    distance_type=distance_type,
    p=p
)

accuracy = accuracy_score(y_test, y_pred)

print("\nDistance Measure:", distance_type)
print("K:", k)

if distance_type == "minkowski":
    print("P:", p)

print(f"Model Accuracy: {accuracy * 100:.2f}%")

```

```

Select Distance Measure:
1. Euclidean
2. Manhattan
3. Minkowski
Enter your choice: 1
Enter value of K: 45

```

Distance Measure: euclidean
K: 45
Model Accuracy: 100.00%

Cell 10: Correct and Wrong Predictions

```
print("\n=== Correct Predictions ===")

for i in range(len(y_test)):
    if y_test[i] == y_pred[i]:
        print(
            f"Sample {i}: "
            f"Predicted = {iris.target_names[y_pred[i]]}, "
            f"Actual = {iris.target_names[y_test[i]]}"
        )

print("\n=== Wrong Predictions ===")

for i in range(len(y_test)):
    if y_test[i] != y_pred[i]:
        print(
            f"Sample {i}: "
            f"Predicted = {iris.target_names[y_pred[i]]}, "
            f"Actual = {iris.target_names[y_test[i]]}"
        )
```

```
=== Correct Predictions ===
Sample 0: Predicted = versicolor, Actual = versicolor
Sample 1: Predicted = setosa, Actual = setosa
Sample 2: Predicted = virginica, Actual = virginica
Sample 3: Predicted = versicolor, Actual = versicolor
Sample 4: Predicted = versicolor, Actual = versicolor
Sample 5: Predicted = setosa, Actual = setosa
Sample 6: Predicted = versicolor, Actual = versicolor
Sample 7: Predicted = virginica, Actual = virginica
Sample 8: Predicted = versicolor, Actual = versicolor
Sample 9: Predicted = versicolor, Actual = versicolor
Sample 10: Predicted = virginica, Actual = virginica
Sample 11: Predicted = setosa, Actual = setosa
Sample 12: Predicted = setosa, Actual = setosa
Sample 13: Predicted = setosa, Actual = setosa
Sample 14: Predicted = setosa, Actual = setosa
Sample 15: Predicted = versicolor, Actual = versicolor
Sample 16: Predicted = virginica, Actual = virginica
Sample 17: Predicted = versicolor, Actual = versicolor
Sample 18: Predicted = versicolor, Actual = versicolor
Sample 19: Predicted = virginica, Actual = virginica
Sample 20: Predicted = setosa, Actual = setosa
Sample 21: Predicted = virginica, Actual = virginica
Sample 22: Predicted = setosa, Actual = setosa
Sample 23: Predicted = virginica, Actual = virginica
Sample 24: Predicted = virginica, Actual = virginica
Sample 25: Predicted = virginica, Actual = virginica
Sample 26: Predicted = virginica, Actual = virginica
Sample 27: Predicted = virginica, Actual = virginica
Sample 28: Predicted = setosa, Actual = setosa
Sample 29: Predicted = setosa, Actual = setosa
```

```
=== Wrong Predictions ===
```

1. Euclidean

Select Distance Measure:

1. Euclidean
2. Manhattan
3. Minkowski

Enter your choice: 1

Enter value of K: 3

2. Manhattan

Enter your choice: 2

Enter value of K: 3

3. Minkowski

Enter your choice: 3

Enter value of K: 3

Enter value of p for Minkowski distance: 3